

1. Ziel und Datengrundlage

Die Anwendung soll einem biologischen Expeditionsteam ermöglichen, neue Pinguinbeobachtungen zu erfassen, einer Art zuzuordnen und dauerhaft zu speichern. Ein grafischer Vergleich mit bekannten Beobachtungen unterstützt die Plausibilitätsprüfung. Die Bedienung erfolgt im Browser und setzt keine Programmierkenntnisse voraus. Dieses Konzept beschreibt das Zielsystem auf Grundlage des vorhandenen Python-Prototyps; insbesondere das Re-Training mit neuen Daten wird in der Implementierungsphase ergänzt.

Grundlage ist der Palmer-Penguins-Datensatz mit 344 Beobachtungen der Arten Adelie, Chinstrap und Gentoo (Horst et al., 2020). In der vorliegenden CSV-Datei fehlen bei zwei Beobachtungen notwendige Körpermesswerte; nach deren Ausschluss verbleiben 342 Datensätze. Fehlendes Geschlecht wird als „unbekannt“ behandelt. Verwendet werden Schnabellänge, Schnabeltiefe, Flossenlänge, Körpermasse und Geschlecht. Die Insel wird als Kontext erfasst, aber nicht zur Klassifikation verwendet. Das Erhebungsjahr entfällt. So stützt sich die Vorhersage auf individuelle Merkmale.

2. Technologien und Architektur

Python mit Dash und Plotly verbindet Eingabeformular, reaktive Ergebnisanzeige und interaktive Diagramme in einer Anwendung. pandas verarbeitet Tabellen, scikit-learn übernimmt Vorverarbeitung und Klassifikation, joblib speichert die trainierte Pipeline. Die Auswahl knüpft an den vorhandenen Projektstand an und hält Entwicklung und Wartung überschaubar. Ein Random Forest mit 500 Bäumen bildet auch nichtlineare Zusammenhänge ab. Numerische Merkmale werden direkt verarbeitet; Geschlechtsangaben werden ergänzt und mittels One-Hot-Encoding kodiert.

Die Architektur trennt Oberfläche und Ablaufsteuerung, Datenaufbereitung, Modellaufbau, Vorhersage mit Validierung, Visualisierung und CSV-Speicherung. Ein separates Trainingsskript führt Training und Bewertung aus. Referenzdaten und Metadaten werden beim Start geladen, die Modellpipeline beim ersten Vorhersageaufruf und danach zwischengespeichert. Der Modellservice prüft die Eingaben und liefert Art, Klassenwahrscheinlichkeiten und gegebenenfalls Warnungen. Erst eine ausdrückliche Speicheraktion schreibt die klassifizierte Beobachtung in die CSV-Datei. Abbildung 2 zeigt die Interaktionen sowie den geplanten Wartungsablauf.

3. Benutzeroberfläche

Abbildung 1 zeigt einen kompakten Entwurf für eine Desktopseite: Links stehen vier Messfelder mit Einheiten, Geschlecht und Insel sowie die Schaltfläche „Pinguinart bestimmen“. Plus- und Minusknöpfe verändern Schnabelmaße um 0,1 mm, Flossenlänge um 1 mm und Körpermasse um 50 g; direkte Eingaben bleiben möglich. Vorgegebene Werte sind als Beispiele erkennbar. Darunter folgen Ergebnis und Speicheraktion mit Statusmeldung. Rechts zeigt ein Streudiagramm die Referenzarten und die neue Beobachtung als Stern. Zwei Auswahlfelder bestimmen die Achsen. Klassenwahrscheinlichkeiten werden als Modellschätzungen erläutert. Änderungen der Eingabe verwerfen das vorherige Ergebnis und erfordern eine neue Klassifikation vor dem Speichern. Auf schmalen Bildschirmen werden die Bereiche untereinander angeordnet.

4. Modellbewertung und kontrollierte Anpassung

Die Bewertung erfolgt mit einem stratifizierten 75/25-Train-Test-Split und fünffacher stratifizierter Kreuzvalidierung auf dem Trainingsanteil. Accuracy, Macro-F1, Cohen's Kappa und Konfusionsmatrix beschreiben Leistung und Fehlerverteilung. Der Zufallsstartwert 42 unterstützt reproduzierbare Vergleiche. Ergebnisse des kleinen Datensatzes sind keine Zusage für neue Expeditionserhebungen. Das nach der Bewertung auf allen verfügbaren Daten trainierte Auslieferungsmodell wird von den separat bewerteten Evaluationsmodellen unterschieden.

Neue Beobachtungen enthalten Messwerte, Insel, Zeitstempel, vorhergesagte Art und Klassenwahrscheinlichkeiten. Das zusätzliche Feld `validated_species` bleibt bis zur fachlichen Bestätigung der tatsächlichen Art leer. Eine Modellvorhersage wird nicht automatisch zum Trainingslabel. Eine verantwortliche Person prüft neue Datensätze, ergänzt bestätigte Arten und startet das Re-Training manuell. Das Trainingsskript soll nur bestätigte Zeilen einlesen, Dubletten prüfen und den bisherigen Trainingsbestand erweitern. Die reproduzierbar festgelegten Testbeobachtungen bleiben beim Training der Evaluationsmodelle ausgeschlossen. Bisherige und neue Variante werden unter gleichen Bedingungen bewertet; eine Übernahme erfolgt nach Prüfung der Kennzahlen. Danach werden Modell und Metadaten gespeichert und die App neu gestartet. Ein kontinuierliches Re-Training ist nicht erforderlich.

5. Plattformunabhängige Bereitstellung

Ein Docker-Image bündelt Anwendung, Modell, Referenzdaten und festgelegte Paketversionen. Nach Installation einer geeigneten Docker-Laufzeit startet die Anwendung lokal und wird im Browser geöffnet. Ein vorbereitetes Image ermöglicht den Betrieb ohne laufende Internetverbindung. Gemäß der Abstimmung mit der Professorin ist kein zusätzlich gehostetes Deployment vorgesehen. Ein eingebundener Datenordner erhält neue CSV-Beobachtungen auch nach dem Entfernen des Containers. CSV genügt für den lokalen Einzelbetrieb. Bei künftigem Mehrbenutzerbetrieb wären koordinierte Schreibzugriffe und gegebenenfalls eine Datenbank zu ergänzen.

6. Datenintegrität und Sicherheit

Messwerte müssen vollständig, numerisch, endlich und positiv sein; Kategorien werden gegen erlaubte Werte geprüft. Werte außerhalb der beobachteten Referenzbereiche erzeugen Hinweise. Fehlerhafte Eingaben und fehlgeschlagene Speichervorgänge werden verständlich gemeldet. Tests prüfen Datenaufbereitung, Modellvorhersage, CSV-Speicherung und die Zuordnung zwischen Eingaben und Ergebnis. Erfasst werden Tierbeobachtungen ohne personenbezogene Angaben. Lokaler Zugriff, geeignete Dateirechte und Sicherungskopien begrenzen unbefugte Änderungen und Datenverlust. Modellartefakte werden nur aus dem eigenen vertrauenswürdigen Projekt geladen. Versionskontrolle und dokumentierte Trainingsläufe machen Änderungen nachvollziehbar.

Quelle: Horst, A. M., Hill, A. P. & Gorman, K. B. (2020). palmerpenguins: Palmer Archipelago (Antarctica) penguin data. <https://allisonhorst.github.io/palmerpenguins/> (06.09.2026). Projektgrundlage: vorhandener Python-Prototyp penguin-classifier-python, insbesondere Datenaufbereitung, Modellpipeline und Trainingsskript; geprüft am 06.09.2026. Abbildungen: KI-unterstützte schematische Entwürfe.

Abbildung 1. Schematischer Oberflächenentwurf

Pinguin-Klassifikator

Messwerte eingeben · Art bestimmen · Ergebnis vergleichen und Beobachtung speichern

1 Messwerte eingeben

Beispielwerte bitte durch eigene Messungen ersetzen.

Schnabellänge (mm)

-	44,5	+
---	------	---

Referenz: 32,1 bis 59,6 mm

Schnabeltiefe (mm)

-	17,3	+
---	------	---

Referenz: 13,1 bis 21,5 mm

Flossenlänge (mm)

-	197	+
---	-----	---

Referenz: 172 bis 231 mm

Körpergewicht (g)

-	4050	+
---	------	---

Referenz: 2700 bis 6300 g

Geschlecht

Unbekannt v

Fundort / Insel (Kontext)

Biscoe v

Pinguinart bestimmen

2 Ergebnis prüfen und speichern

Art + Modellwahrscheinlichkeiten + ggf. Warnung

Beobachtung speichern

Statusmeldung

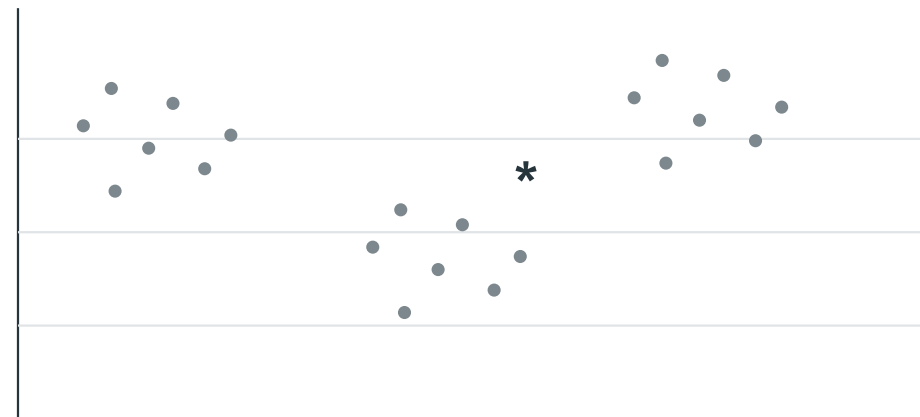
3 Mit Referenzdaten vergleichen

X-Achse

Schnabellänge (mm) v

Y-Achse

Schnabeltiefe (mm) v

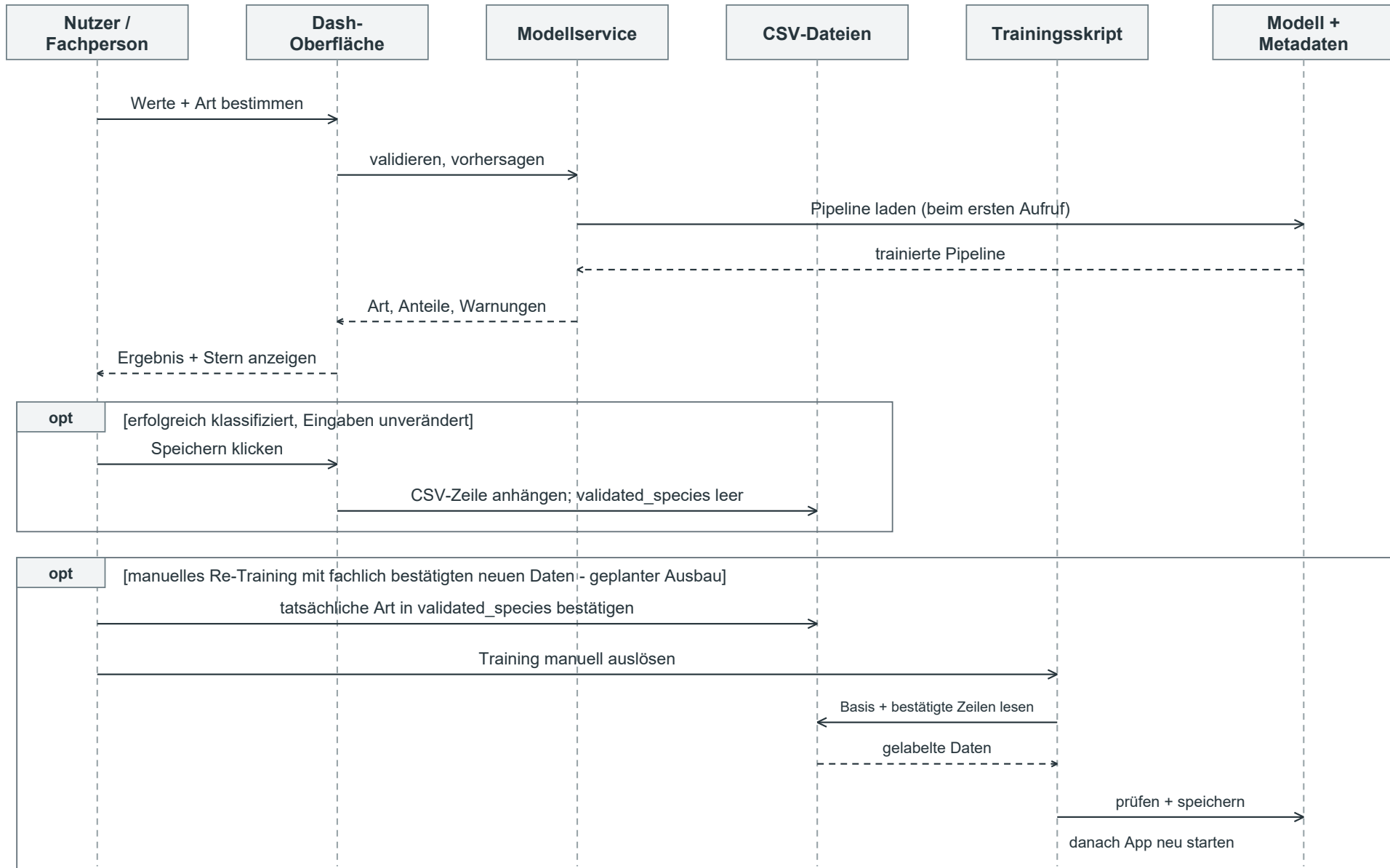


gewähltes Körpermerkmal

Art-Legende · Stern = neue klassifizierte Beobachtung

Schematischer Entwurf, keine Messdatendarstellung. Desktop: Eingabe, Ergebnis und Diagramm nebeneinander. Auf schmalen Geräten werden die Bereiche untereinander angeordnet.

Abbildung 2. UML-Sequenzdiagramm



Fehlerfall: Ungültige Eingaben liefern eine Fehlermeldung; Klassifikation und Speichern bleiben aus. Änderungen der Eingabe verwerfen das vorherige Ergebnis. Gestrichelt: Rückgabe / Lebenslinie.